Robust Zero-Sum RL with Transformer Disturbance Detection for Embedded Control

Hadi Nobahari, Ali Baniasad
Department of Aerospace Engineering, Sharif University of Technology, Tehran, Iran
Email: nobahari@sharif.ir, ali_baniasad@ae.sharif.edu

***Index Terms*—robust reinforcement learning, zero-sum Markov games, adversarial training, transformer anomaly detection, policy mixing, TensorRT, embedded deployment**

Abstract*—Deploying reinforcement learning (RL) policies on physical robots requires robustness to unmodeled disturbances, parameter uncertainty, and the sim-to-real gap. We present a methodology for robust control that frames the problem as a two-player zero-sum Markov game in which a learned adversary systematically probes the weaknesses of the control policy during training. Our architecture maintains two distinct policies: an *optimal* policy π^{opt} trained on the nominal environment to maximize expected return, and a *robust* policy π^{rob} co-trained against the adversary to maximize worst-case return. At runtime, a lightweight transformer encoder processes a sliding window of recent observations to produce a real-time disturbance estimate—both a detection probability and an estimated magnitude. A learned gating function converts this estimate into a continuous mixing coefficient $\alpha_t \in [0, 1]$ that blends the two policies, yielding actions $a_t = \alpha_t \, \pi^{\text{rob}}(s_t) + (1 - \alpha_t) \, \pi^{\text{opt}}(s_t)$. The training pipeline is implemented in PyTorch with Gymnasium and MuJoCo, with a planned migration to JAX and MJX for GPU-vectorized environment parallelism. For deployment, all neural network components are exported through ONNX to TensorRT engines optimized for the NVIDIA Jetson platform, and integrated into a ROS 2 control node that achieves sub-two-millisecond inference latency at up to 500 Hz. We provide a complete data-logging specification, three detailed algorithmic procedures covering adversarial training, transformer detector training, and real-time inference, and a comprehensive evaluation plan with ablation studies and baselines. Preliminary experiments on the Ant-v5 benchmark demonstrate that our method achieves the most consistent robustness ratios across four disturbance scenarios compared to Vanilla DDPG, RARL, SA-MDP, and domain randomization baselines, while maintaining the longest survival times under all tested disturbance conditions.

I. Introduction

Reinforcement learning has achieved impressive results in simulated continuous-control benchmarks, yet transferring learned policies to physical robots remains a significant challenge. Real-world systems are subject to disturbances—external forces, friction changes, payload variation, sensor noise, and actuator degradation—that are difficult to capture faithfully in a simulator. The resulting *sim-to-real gap* frequently causes policies that perform well in simulation to fail when deployed on hardware [1].

A principled approach to this problem is *robust* reinforcement learning, which seeks policies that perform well not only under nominal conditions but also under worst-case perturbations. One particularly elegant formulation models the interaction between the controller and the environment disturbance as a *two-player zero-sum Markov game* [2]: the controller maximizes cumulative reward while a learned adversary minimizes it, and training converges toward a Nash equilibrium that provides formal worst-case guarantees. This approach, pioneered by Robust Adversarial Reinforcement Learning (RARL) [3] and the H_∞ control perspective [4], has since been extended to state-adversarial formulations [5] and adversarial populations [6].

Despite these advances, several gaps remain in the literature. First, existing adversarial training methods produce a *single* policy that compromises between nominal performance and worst-case robustness; no mechanism exists to recover full nominal performance when disturbances are absent. Second, at deployment time, the policy has no explicit *awareness* of whether or not a disturbance is currently active: it applies the same conservative strategy regardless of conditions. Third, transformer-based anomaly detection has shown strong results in multivariate time-series domains [7], [8], yet this capability has not been leveraged for runtime disturbance detection in RL control systems. Finally, few works address the complete pipeline from adversarial training through model export to real-time inference on embedded GPU platforms with ROS 2 integration.

This paper addresses these gaps with the following contributions:

- A **dual-policy mixing architecture** that maintains separate optimal and robust policies and blends their actions through a continuous gating coefficient, preserving nominal performance while providing worst-case robustness on demand.
- A **transformer-based disturbance detector** that operates on a sliding window of observations and outputs a detection probability and estimated disturbance magnitude in real time, providing the signal needed for the gating decision.
- A **unified training pipeline** with a detailed data-logging specification that supports both off-policy and on-policy adversarial training algorithms and generates the supervised dataset for the transformer detector.
- A **deployment pipeline** that exports all neural network components to ONNX, optimizes them with TensorRT for the NVIDIA Jetson platform, and integrates them into a ROS 2 node meeting sub-two-millisecond latency budgets.

- A **comprehensive evaluation plan** including ablation studies, baselines, and safety checks, designed to rigorously validate the approach once empirical experiments are conducted.

The remainder of this paper is organized as follows. Section II reviews the necessary technical background. Section III surveys related work. Section IV formalizes the problem. Section V presents the method in detail, including algorithms and data specifications. Section VI describes the system implementation. Section VII presents the experimental results. Section VIII discusses limitations, and Section IX concludes.

II. Background and Preliminaries

A. Markov Decision Processes

A Markov Decision Process (MDP) is defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ is the transition kernel, $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function, and $\gamma \in [0, 1)$ is the discount factor. A stochastic policy $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ maps states to distributions over actions. The state-value function and action-value function under policy π are

$$V^\pi(s) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \,\middle|\, s_0 = s\right], \tag{1}$$

$$Q^\pi(s, a) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \,\middle|\, s_0 = s, a_0 = a\right]. \tag{2}$$

These satisfy the Bellman equation

$$V^\pi(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}\Big[R(s, a) + \gamma\, \mathbb{E}_{s'\sim P(\cdot|s,a)} V^\pi(s')\Big]. \tag{3}$$

The goal of reinforcement learning is to find a policy π^* that maximizes $V^\pi(s)$ for all $s \in \mathcal{S}$.

B. Two-Player Zero-Sum Markov Games

A two-player zero-sum Markov game [2] extends the MDP to $(\mathcal{S}, \mathcal{A}_1, \mathcal{A}_2, P, R, \gamma)$, where $\mathcal{A}_1$ and $\mathcal{A}_2$ are the action spaces of Player 1 (the controller) and Player 2 (the adversary/disturbance), respectively. The transition kernel becomes $P : \mathcal{S} \times \mathcal{A}_1 \times \mathcal{A}_2 \times \mathcal{S} \to [0, 1]$, and the reward $R(s, a_1, a_2)$ is received by the controller while the adversary receives $-R(s, a_1, a_2)$. A pair of policies (π_1^*, π_2^*) constitutes a *Nash equilibrium* if

$$V^{\pi_1^*,\pi_2}(s) \;\geq\; V^{\pi_1^*,\pi_2^*}(s) \;\geq\; V^{\pi_1,\pi_2^*}(s) \tag{4}$$

for all s, π_1, π_2. The *minimax value* is

$$V^*(s) = \max_{\pi_1} \min_{\pi_2}\; \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t^1, a_t^2) \,\middle|\, s_0 = s\right]. \tag{5}$$

This formulation provides a natural framework for robust control: the controller learns to maximize reward against the strongest possible adversary, yielding a policy with formal worst-case performance guarantees.

C. Deep Reinforcement Learning Algorithms

We consider four deep RL algorithms as candidate training methods for both the controller and the adversary.

DDPG [9] is an off-policy actor-critic algorithm for continuous action spaces. It maintains a deterministic policy $\mu_\theta(s)$, a critic $Q_\phi(s, a)$, a replay buffer for experience storage, and target networks $\mu_{\theta'}$, $Q_{\phi'}$ updated via Polyak averaging. The critic is trained by minimizing the temporal-difference (TD) error, and the actor is updated by applying the deterministic policy gradient [10].

TD3 [11] improves upon DDPG with three modifications: (i) twin critics Q_{ϕ_1}, Q_{ϕ_2} with the minimum taken as the target (reducing overestimation bias), (ii) delayed policy updates (the actor is updated less frequently than the critic), and (iii) target policy smoothing (noise is added to the target action to regularize the critic).

SAC [12] is an off-policy algorithm based on the maximum-entropy RL framework. The objective augments the expected return with an entropy term:

$$J_{\text{SAC}}(\pi) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t \Big(R(s_t, a_t) + \beta\, \mathcal{H}\big(\pi(\cdot|s_t)\big)\Big)\right], \tag{6}$$

where $\beta > 0$ is the temperature parameter (often automatically tuned) and $\mathcal{H}$ denotes the Shannon entropy. SAC learns a stochastic policy parameterized via the reparameterization trick, combined with twin critics as in TD3.

PPO [13] is an on-policy algorithm that optimizes a clipped surrogate objective:

$$J_{\text{PPO}}(\theta) = \mathbb{E}_t\Big[\min\big(r_t(\theta)\hat{A}_t,\; \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\Big], \tag{7}$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio and $\hat{A}_t$ is the generalized advantage estimate (GAE). PPO collects rollouts in batches and performs multiple epochs of mini-batch optimization before discarding the data.

D. Robust Reinforcement Learning Formulations

Several formulations address robustness in RL. *Robust MDPs* [14], [15] replace the single transition kernel P with an *uncertainty set* $\mathcal{P} \ni P$, and the agent optimizes against the worst-case element: $V^*(s) = \max_\pi \min_{P\in\mathcal{P}} V_P^\pi(s)$. The H_∞ *approach* [4] penalizes the sensitivity of the value function to disturbances, yielding a minimax objective similar to (5) but with a continuous disturbance signal rather than a discrete adversary.

Adversarial training (RARL) [3] operationalizes the zero-sum game by training a neural-network adversary that applies forces to the agent's body, alternating between controller and adversary updates. This has been extended to *state-adversarial* MDPs [5], where the adversary perturbs the observations rather than the dynamics, and *action-robust* formulations [16], where the adversary modifies the executed action. A comprehensive survey of robust RL methods is provided by Moos et al. [17].

III. Related Work

A. Adversarial and Robust Reinforcement Learning

The idea of training RL agents against adversarial perturbations originates with Morimoto and Doya [4], who cast robust control as a differential game and solved it with actor-critic methods. Pinto et al. [3] introduced RARL, in which a destabilizing adversary applies forces to the agent's body during training, demonstrating improved transfer to environments with modified physical parameters. Pan et al. [18] proposed a risk-averse variant of RARL that incorporates CVaR-based objectives to further reduce tail risk. Vinitsky et al. [6] showed that training against an *ensemble* of adversarial policies, rather than a single adversary, prevents the controller from overfitting to one attack strategy and improves generalization.

Zhang et al. [5] formulated the state-adversarial MDP (SA-MDP), in which the adversary perturbs the agent's observations within a bounded ℓ_p ball. They derived a regularizer that encourages policy smoothness with respect to state perturbations. Oikarinen et al. [19] extended this idea by incorporating a certified adversarial loss during training, providing provable robustness bounds for linear function approximators. Sun et al. [20] studied the problem of finding *optimal* adversarial attacks against deep RL agents, characterizing the strongest possible attack under budget constraints and showing that many existing attack heuristics are suboptimal.

A comprehensive survey by Moos et al. [17] categorizes robust RL methods along several axes: uncertainty sets, adversary types, and solution concepts. A common observation across this body of work is that adversarial training is performed entirely at *training time*: the resulting policy applies the same robust strategy at every timestep during deployment, regardless of whether a disturbance is actually present. This conflates nominal and adversarial operation, sacrificing performance when conditions are benign. Our work addresses this gap by introducing a runtime disturbance detection mechanism that gates between nominal-optimal and adversarially-robust policies.

B. Transformers in Time-Series and Reinforcement Learning

The transformer architecture [21], originally developed for natural language processing, has been successfully applied to time-series analysis and anomaly detection. TranAD [7] uses an adversarially-trained transformer with attention-based reconstruction for detecting anomalies in multivariate sensor streams, achieving state-of-the-art results on industrial monitoring benchmarks. The Anomaly Transformer [8] introduces an *association discrepancy* mechanism that distinguishes anomalous timesteps from normal ones by comparing learned associations with a prior distribution.

In the RL domain, Chen et al. [22] proposed the Decision Transformer, which formulates RL as a sequence-modeling problem and conditions action generation on desired return-to-go, states, and actions. While this work demonstrates the power of transformers for representing sequential decision-making, it does not address disturbance detection.

To our knowledge, no prior work uses transformers specifically for *runtime disturbance detection* in the context of robust RL, nor does any work use such a detector to gate between separate optimal and robust control policies.

C. Sim-to-Real Transfer and Deployment

Sim-to-real transfer remains a central challenge in robot learning. *Domain randomization* [23] trains policies on a distribution of environment parameters (visual appearance, physical properties) so that the real world appears as just another sample from the training distribution. Peng et al. [24] demonstrated that randomizing dynamics parameters during training—mass, friction, damping—enables zero-shot transfer of manipulation policies. Zhao et al. [1] provide a survey of sim-to-real transfer methods, covering domain randomization, system identification, and progressive adaptation.

Lee et al. [25] used a teacher-student framework for quadrupedal locomotion: a privileged teacher trained in simulation distills its knowledge into a student that relies only on proprioceptive observations available on the real robot. This approach has become a standard paradigm in legged locomotion.

On the simulation side, MuJoCo [26] remains the dominant physics engine for contact-rich control. The recent MJX extension [27] provides a JAX-based implementation of MuJoCo that enables massively parallel environment simulation on GPUs. Brax [28] offers a similar JAX-based differentiable physics engine. Gymnasium [29] provides the standard Python interface for RL environments.

D. Embedded Inference for Real-Time Control

Deploying neural-network policies on embedded platforms requires careful attention to inference latency and throughput. NVIDIA's TensorRT [30] is a high-performance inference optimizer that applies layer fusion, kernel auto-tuning, precision calibration (FP16, INT8), and memory optimization to minimize latency on NVIDIA GPUs. Jeon et al. [31] developed a framework and optimization methodology specifically for TensorRT inference on Jetson boards, achieving significant speedups over native PyTorch execution.

The ONNX ecosystem [32] provides a standard interchange format for neural networks, enabling models trained in any framework (PyTorch, TensorFlow, JAX) to be converted to a common representation and subsequently optimized by TensorRT. This is particularly important for transformer architectures, where INT8 quantization of attention layers requires careful calibration.

NVIDIA's Isaac ROS [33] provides GPU-accelerated ROS 2 packages built on the NITROS framework, which enables zero-copy GPU tensor transfer between ROS nodes. This infrastructure is critical for achieving the low latencies required by real-time control.

Despite the maturity of individual components, few works address the *complete pipeline* from adversarial RL training in simulation through ONNX export and TensorRT optimization to real-time deployment on Jetson hardware with ROS 2

TABLE I: Feature comparison with related approaches.

Feature	RARL	SA-MDP	DR	Ours
Adversarial training	✓	✓	–	✓
Dual-policy mixing	–	–	–	✓
Runtime detection	–	–	–	✓
Transformer detector	–	–	–	✓
TensorRT deployment	–	–	–	✓
ROS 2 integration	–	–	–	✓

integration. Our work provides a detailed design for this end-to-end pipeline.

E. Comparison to Our Approach

Table I summarizes how our approach relates to prior work. The key differentiators are: (i) the dual-policy architecture with continuous gating, (ii) the use of a transformer for runtime disturbance detection to drive the gating signal, and (iii) the end-to-end deployment pipeline targeting embedded GPUs with real-time ROS 2 integration. No prior work combines all three elements.

IV. Problem Formulation

A. Two-Player Zero-Sum Markov Game

We consider a two-player zero-sum Markov game defined by the tuple $(\mathcal{S}, \mathcal{A}^c, \mathcal{A}^d, P, R, \gamma)$, where $\mathcal{S} \subseteq \mathbb{R}^n$ is the continuous state space, $\mathcal{A}^c \subseteq \mathbb{R}^{m_c}$ is the controller's action space, and $\mathcal{A}^d \subseteq \mathbb{R}^{m_d}$ is the disturbance (adversary) action space. The transition dynamics are governed by $P(s' \mid s, a^c, a^d)$, where $s \in \mathcal{S}$, $a^c \in \mathcal{A}^c$, $a^d \in \mathcal{A}^d$, and $s' \in \mathcal{S}$. The scalar reward $r(s, a^c, a^d) \in \mathbb{R}$ is received by the controller; the adversary receives $-r(s, a^c, a^d)$.

The controller seeks a policy $\pi^c : \mathcal{S} \to \Delta(\mathcal{A}^c)$ that maximizes the expected discounted return, while the adversary seeks a policy $\pi^d : \mathcal{S} \to \Delta(\mathcal{A}^d)$ that minimizes it. A Nash equilibrium $(\pi^{c,*}, \pi^{d,*})$ satisfies

$$V^{\pi^{c,*},\pi^d}(s) \geq V^{\pi^{c,*},\pi^{d,*}}(s) \geq V^{\pi^c,\pi^{d,*}}(s) \tag{8}$$

for all $s \in \mathcal{S}$, π^c, π^d. The minimax value function is

$$V^*(s) = \max_{\pi^c} \min_{\pi^d} \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \, r(s_t, a_t^c, a_t^d) \,\middle|\, s_0 = s\right]. \tag{9}$$

B. Disturbance Model

The adversary's action $a^d \in \mathcal{A}^d$ represents an external disturbance applied to the system. We consider two concrete instantiations:

1) **Additive force/torque perturbation.** The adversary applies a force or torque vector to one or more joints or links of the robot. The perturbed dynamics become $\dot{q} = f(q, \dot{q}, a^c) + B\, a^d$, where $B \in \mathbb{R}^{n_q \times m_d}$ is a known input matrix selecting the affected degrees of freedom.
2) **External body force.** The adversary applies an external wrench (force and torque) to a specified body, modeling pushes, wind, or contact with unmodeled objects.

In both cases, the adversary's action is bounded:

$$\|a^d\|_2 \leq \epsilon, \tag{10}$$

where $\epsilon > 0$ is the *disturbance budget*. The adversary learns to choose the direction and magnitude of the perturbation within this budget to maximally degrade the controller's performance. The disturbance can be further parameterized by a magnitude $\delta \in [0, \delta_{\max}]$ and a unit direction vector $\boldsymbol{d} \in \mathbb{R}^{m_d}$, $\|\boldsymbol{d}\|_2 = 1$, so that $a^d = \delta\, \boldsymbol{d}$.

C. Objective: Dual-Policy Robust Control

Rather than training a single policy that compromises between nominal performance and worst-case robustness, we maintain two separate policies:

- π^{opt}: the *optimal* policy, trained on the nominal environment (no adversary) to maximize $\mathbb{E}[\sum_t \gamma^t r_t]$.
- π^{rob}: the *robust* policy, co-trained with the adversary π^{adv} in the zero-sum game to maximize $\min_{\pi^d} \mathbb{E}[\sum_t \gamma^t r_t]$.

Both policies share the same observation and action spaces. At runtime, the executed action is a convex combination:

$$a_t = \alpha_t \cdot \pi^{\text{rob}}(s_t) + (1 - \alpha_t) \cdot \pi^{\text{opt}}(s_t), \tag{11}$$

where the mixing coefficient $\alpha_t \in [0, 1]$ is produced by a *gating network* g_θ operating on a history of observations:

$$\alpha_t = g_\theta(h_t), \quad h_t = (o_{t-L+1},\, o_{t-L+2},\, \ldots,\, o_t), \tag{12}$$

with $o_i \in \mathbb{R}^d$ denoting the observation at timestep i and L denoting the history window length.

When $\alpha_t \approx 0$, the system executes the nominal-optimal policy; when $\alpha_t \approx 1$, it executes the robust policy. This design preserves the full nominal performance of π^{opt} in undisturbed conditions while providing the worst-case guarantees of π^{rob} when disturbances are detected.

D. Real-Time Constraints

Real-time control imposes strict latency requirements. Let τ_{budget} denote the maximum allowable inference time per control step. For a control frequency of f Hz, we require

$$\tau_{\text{gate}} + \tau_{\text{policy}} + \tau_{\text{pub}} \leq \tau_{\text{budget}} = \frac{1}{f}. \tag{13}$$

Here τ_{gate} is the time for the transformer forward pass and gating computation, τ_{policy} is the time for the two policy forward passes (which may be pipelined), and τ_{pub} is the time for ROS message serialization and publication. For the target range of $f \in [200, 500]$ Hz, this yields $\tau_{\text{budget}} \in [2, 5]$ ms. All neural network components must be optimized to meet this budget on the NVIDIA Jetson platform.

V. Method

This section presents the proposed approach in detail. We describe the dual-policy architecture, the adversary training objective, the transformer disturbance detector, the gating rule, the data-logging specification, and the complete algorithmic procedures.

A. Dual-Policy Architecture

The optimal policy π^{opt} and the robust policy π^{rob} are trained independently. Both are parameterized as feedforward neural networks (typically two hidden layers of 256 units with ReLU activations) that map observations $o \in \mathbb{R}^d$ to actions $a \in \mathbb{R}^{m_c}$.

Optimal policy training. The policy π^{opt} is trained using a standard deep RL algorithm (SAC, TD3, or PPO) on the nominal environment, i.e., with the adversary disabled ($a^d = \mathbf{0}$). This policy maximizes the expected return under nominal dynamics and represents the best achievable performance when no disturbance is present.

Robust policy training. The policy π^{rob} is trained in the zero-sum game against a co-learned adversary π^{adv}. The controller and adversary are updated in alternation, as described in Section V-B. Upon convergence, π^{rob} approximates the maximin strategy and provides robust performance under the worst-case disturbance within the budget ϵ.

Rationale for separate training. A single policy trained with adversarial perturbations necessarily trades off nominal performance for robustness: it learns conservative actions even in the absence of disturbances. By maintaining two separate policies, our architecture avoids this trade-off. The gating mechanism (Section V-D) selects between them based on the detected disturbance level, achieving near-optimal nominal performance and near-optimal worst-case performance simultaneously.

Architecture overview. Figure 1 illustrates the complete runtime architecture. A sliding window of recent observations is processed by the transformer detector to produce disturbance estimates, which drive the gating network to compute the mixing coefficient α_t. The final action is a continuous blend of the optimal and robust policies.

B. Adversary Training Objective

The adversary π^{adv} is a neural network with the same architecture as the controller policies, mapping observations $o \in \mathbb{R}^d$ to disturbance actions $a^d \in \mathbb{R}^{m_d}$. The output layer uses a $\tanh$ activation scaled by the disturbance budget ϵ:

$$a^d = \epsilon \cdot \tanh\big(\text{NN}_\psi(o)\big), \tag{14}$$

which ensures $\|a^d\|_\infty \leq \epsilon$ by construction.

The adversary's objective is to minimize the controller's return, i.e., to maximize the negated return:

$$J^{\text{adv}}(\pi^{\text{adv}}) = -J^c(\pi^{\text{rob}}, \pi^{\text{adv}}) = -\mathbb{E}_{\pi^{\text{rob}}, \pi^{\text{adv}}}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t^c, a_t^d)\right]. \tag{15}$$

Training proceeds by *alternating optimization*:

1) Fix π^{adv}; update π^{rob} to maximize $J^c(\pi^{\text{rob}}, \pi^{\text{adv}})$ using the chosen RL algorithm.
2) Fix π^{rob}; update π^{adv} to maximize $J^{\text{adv}}(\pi^{\text{adv}})$ using the same (or a separate) RL algorithm.

This alternation is related to *fictitious play* in game theory and to the *double oracle* method for solving zero-sum games. In practice, both players share a single replay buffer (for off-policy methods) and are updated with a fixed ratio of gradient steps per environment step.

To prevent the adversary from becoming too strong too quickly (which can destabilize the controller's learning), we employ curriculum scheduling of the disturbance budget: ϵ is linearly increased from 0 to $\epsilon_{\max}$ over the first N_{warmup} episodes.

Figure 2 illustrates the two-phase training procedure and the data flow through the system.

C. Transformer Disturbance Detector

The disturbance detector is a lightweight transformer encoder that processes a sliding window of recent observations and produces two outputs: a disturbance detection probability $p_t \in [0, 1]$ and an estimated disturbance magnitude $\hat{\delta}_t \geq 0$.

Input representation. The input to the transformer is a sequence of L observation vectors $h_t = (o_{t-L+1}, \ldots, o_t)$, where each $o_i \in \mathbb{R}^d$. The sequence is treated as a matrix $\boldsymbol{H}_t \in \mathbb{R}^{L \times d}$.

Positional encoding. Since the transformer architecture is permutation-invariant, we add sinusoidal positional encodings to each element of the sequence:

$$\text{PE}(k, 2j) = \sin\left(\frac{k}{10000^{2j/d_{\text{model}}}}\right), \tag{16}$$

$$\text{PE}(k, 2j+1) = \cos\left(\frac{k}{10000^{2j/d_{\text{model}}}}\right), \tag{17}$$

where $k \in \{0, \ldots, L-1\}$ is the position index and $j \in \{0, \ldots, d_{\text{model}}/2 - 1\}$ indexes the dimension.

Architecture. The encoder consists of $N_L \in \{2, 3, 4\}$ transformer layers, each containing multi-head self-attention with $N_H \in \{2, 4\}$ heads and a position-wise feedforward network. The model dimension is $d_{\text{model}} \in \{64, 128\}$ and the feedforward dimension is $d_{\text{ff}} = 4 \cdot d_{\text{model}}$. Layer normalization is applied before each sub-layer (pre-norm configuration). A *causal attention mask* is applied so that each timestep attends only to current and past observations, ensuring the model can operate in a streaming fashion at deployment.

Output head. The transformer output at the final position (timestep t) is passed through a small MLP with two output branches:

- Detection branch: linear layer $\to$ sigmoid, producing $p_t \in [0, 1]$.
- Magnitude branch: linear layer $\to$ softplus, producing $\hat{\delta}_t \geq 0$.

Training. The transformer is trained in a supervised fashion on data logged during the adversarial training phase (Section V-E). For each window $(o_{t-L+1}, \ldots, o_t)$, the label consists of the ground-truth disturbance indicator $y_t \in \{0, 1\}$ (whether a disturbance was applied at timestep t) and the ground-truth disturbance magnitude $\delta_t = \|a_t^d\|_2$. The training loss is

$$\mathcal{L} = \mathcal{L}_{\text{BCE}}(p_t, y_t) + \lambda\, \mathcal{L}_{\text{MSE}}(\hat{\delta}_t, \delta_t), \tag{18}$$

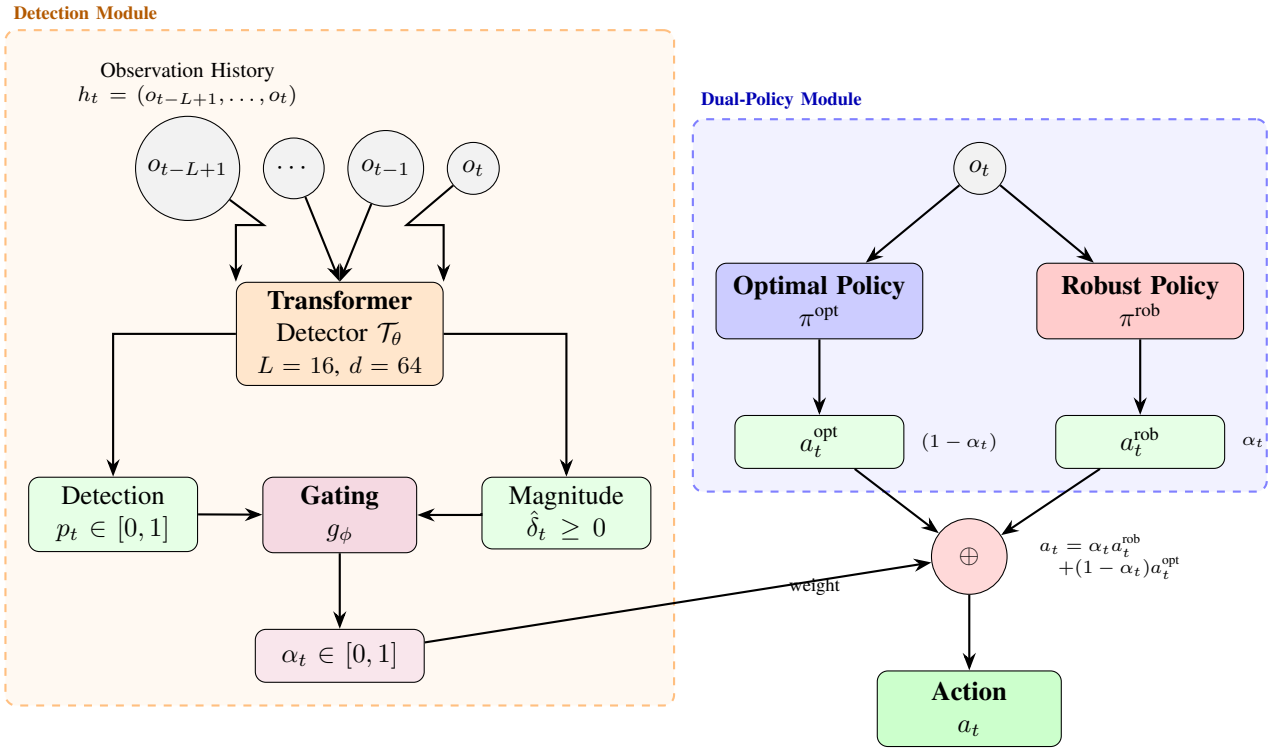

Fig. 1: Runtime architecture of the proposed method. The transformer detector processes a sliding window of observations to estimate disturbance probability and magnitude, which drive the gating network to compute the mixing coefficient α_t. The final action is a continuous blend of the optimal policy (trained on nominal environment) and the robust policy (trained against adversary), preserving nominal performance while providing worst-case robustness on demand.

where $\mathcal{L}_{\text{BCE}}$ is the binary cross-entropy loss, $\mathcal{L}_{\text{MSE}}$ is the mean squared error, and $\lambda > 0$ is a weighting hyperparameter balancing the two objectives.

Design for deployment. The transformer is intentionally kept small to satisfy the real-time latency budget on Jetson hardware. With $N_L = 2$, $d_{\text{model}} = 64$, $N_H = 2$, and $L = 16$, the model has approximately 50k parameters and can be executed in under 0.5 ms on a Jetson Orin after TensorRT optimization.

D. Policy Mixing and Gating Rule

The gating function converts the transformer's outputs into the mixing coefficient α_t. We consider two variants:

Linear gating. A simple parameterization:

$$\alpha_t = \sigma(w_p \cdot p_t + w_\delta \cdot \hat{\delta}_t + b), \tag{19}$$

where $\sigma(\cdot)$ is the logistic sigmoid, and w_p, w_δ, b are learnable scalar parameters.

MLP gating. A more expressive variant:

$$\alpha_t = f_\phi(p_t, \hat{\delta}_t), \tag{20}$$

where f_ϕ is a small multi-layer perceptron (e.g., two hidden layers of 32 units). The MLP can learn nonlinear relationships such as threshold effects or hysteresis-like behavior.

In both cases, the final action is computed as

$$a_t = \alpha_t \, \pi^{\text{rob}}(s_t) + (1 - \alpha_t) \, \pi^{\text{opt}}(s_t). \tag{21}$$

Soft mixing vs. hard switching. An alternative design uses hard switching: $\alpha_t \in \{0, 1\}$ based on a threshold on p_t. While simpler, hard switching introduces *chattering*—rapid oscillation between policies near the decision boundary—which can produce discontinuous control signals. Soft mixing with a continuous α_t avoids this problem and allows smooth transitions between the two policies. We include a comparison of soft and hard switching in the ablation study (Section VII-H).

E. Data Logging Specification

A standardized data-logging format is essential for reproducibility and for generating the supervised training set for the transformer detector. Table II specifies the fields logged at every timestep during training.

Sequence window extraction. To train the transformer detector, we extract sliding windows of length L from the logged episodes. For timesteps $t < L$ within an episode, the window is left-padded with zeros and an attention mask is constructed so that the transformer ignores padded positions. Windows *never* cross episode boundaries: if the remaining timesteps in an episode are fewer than L, the window is zero-padded from the left rather than drawing observations from the next episode.

Storage format. Data is stored in HDF5 files (one per training run) with chunked and compressed datasets, or in

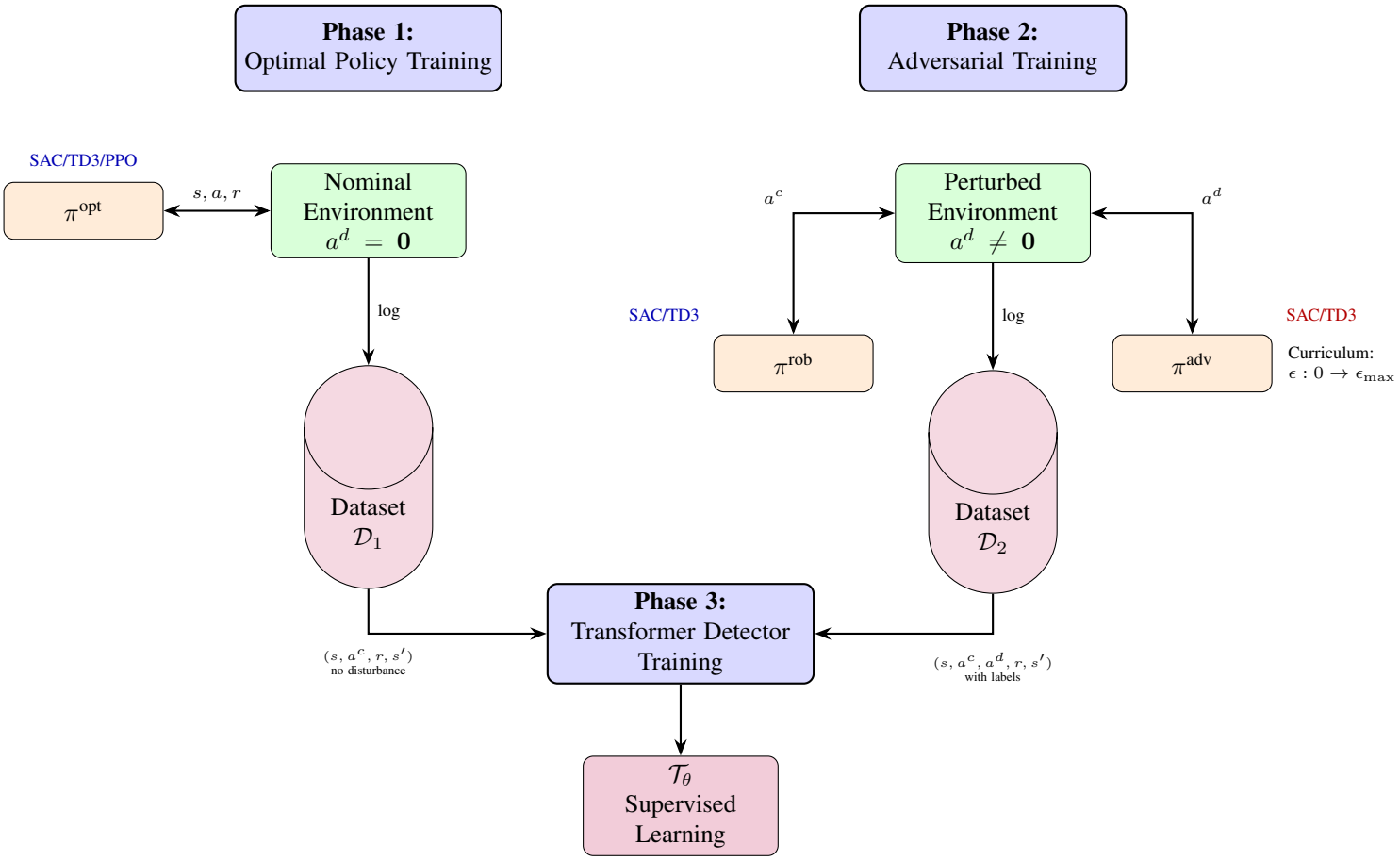

Fig. 2: Training pipeline overview. Phase 1 trains the optimal policy on the nominal environment. Phase 2 co-trains the robust policy and adversary in a zero-sum game with curriculum scheduling of the disturbance budget. Both phases log complete trajectories. Phase 3 trains the transformer detector via supervised learning on the combined dataset with ground-truth disturbance labels.

TABLE II: Data logging specification. Each row is recorded per timestep.

Field	Type	Description
`t`	`int`	Timestep index within episode
`episode_id`	`int`	Unique episode identifier
`obs`	`float[`n`]`	Observation vector
`action_ctrl`	`float[`m_c`]`	Controller action
`action_dist`	`float[`m_d`]`	Disturbance/adversary action
`action_total`	`float[`m_c`]`	Applied action after mixing
`reward`	`float`	Scalar reward
`terminated`	`bool`	Episode termination flag
`truncated`	`bool`	Episode truncation flag
`info/dist_params`	`float[`k`]`	Ground-truth disturbance parameters
`disturbance_tag`	`str`	Categorical disturbance label
`robust_weight`	`float`	Target α_t for gating supervision

Apache Arrow/Parquet format for efficient columnar access. Each episode is a separate group (HDF5) or partition (Arrow), enabling efficient random access during window extraction.

F. Algorithm Pseudocode

We present three algorithms covering the complete pipeline: adversarial training, transformer detector training, and deployment inference.

Algorithm 1 describes the two-phase off-policy training loop. Phase 1 trains the optimal policy without any adversary. Phase 2 trains the robust policy against the adversary, with curriculum scheduling of the disturbance budget. Both phases log all transitions to the dataset $\mathcal{D}$.

Algorithm 2 describes the supervised training of the transformer disturbance detector using the logged data.

pa-Algorithm 3 describes the real-time inference loop running as a ROS 2 node on the Jetson platform.

VI. System Implementation

A. Buffer and Logger Utilities

The system uses two distinct buffer types depending on the training algorithm.

Replay buffer (off-policy). A fixed-capacity circular buffer stores transitions (s, a^c, a^d, r, s', d). The buffer supports uniform random sampling and, optionally, Prioritized Experience

Algorithm 1 Off-Policy Adversarial Training Loop

Input: Environment env; actor networks π^{opt}, π^{rob}; critic Q_ϕ; adversary π^{adv}; replay buffer $\mathcal{B}$; dataset logger $\mathcal{D}$; episodes N; budget schedule $\epsilon(\cdot)$
Output: Trained π^{opt}, π^{rob}, π^{adv}; dataset $\mathcal{D}$

Phase 1: Train optimal policy
for episode $= 1$ **to** N_{opt} **do**
  $s_0 \leftarrow$ env.reset()
  for $t = 0, 1, \ldots, T-1$ **do**
    $a_t^c \leftarrow \pi^{\text{opt}}(s_t) + \text{noise}$
    $s_{t+1}, r_t, d_t \leftarrow \text{env.step}(a_t^c, \mathbf{0})$
    $\mathcal{B}$.add$(s_t, a_t^c, r_t, s_{t+1}, d_t)$
    $\mathcal{D}$.log$(t, s_t, a_t^c, \mathbf{0}, r_t, d_t)$
    Update π^{opt}, Q_ϕ from $\mathcal{B}$ (SAC/TD3)
  end for
end for
Phase 2: Train robust policy with adversary
for episode $= 1$ **to** N **do**
  $\epsilon_t \leftarrow \epsilon(\text{episode})$ ▷ Curriculum
  $s_0 \leftarrow$ env.reset()
  for $t = 0, 1, \ldots, T-1$ **do**
    $a_t^c \leftarrow \pi^{\text{rob}}(s_t) + \text{noise}$
    $a_t^d \leftarrow \pi^{\text{adv}}(s_t)$, clipped to $\|a_t^d\| \le \epsilon_t$
    $s_{t+1}, r_t, d_t \leftarrow \text{env.step}(a_t^c, a_t^d)$
    $\mathcal{B}$.add$(s_t, a_t^c, a_t^d, r_t, s_{t+1}, d_t)$
    $\mathcal{D}$.log$(t, s_t, a_t^c, a_t^d, r_t, d_t)$
    Sample batch from $\mathcal{B}$
    Update Q_ϕ: minimize TD error
    Update π^{rob}: maximize $Q_\phi(s, \pi^{\text{rob}}(s))$
    Update π^{adv}: minimize $Q_\phi(s, \pi^{\text{rob}}(s), \pi^{\text{adv}}(s))$
  end for
end for

Algorithm 2 Transformer Disturbance Detector Training

Input: Dataset $\mathcal{D}$ (logged episodes); window length L; transformer model $\mathcal{T}_\theta$; learning rate η; loss weight λ; epochs E
Output: Trained detector $\mathcal{T}_\theta$

Extract sliding windows $\{(h_i, y_i, \delta_i)\}_{i=1}^{M}$ from $\mathcal{D}$
  $h_i = (o_{i-L+1}, \ldots, o_i)$, with zero-padding and attention masks
  $y_i = \mathbb{1}[\|a_i^d\|_2 > 0], \quad \delta_i = \|a_i^d\|_2$
Split into training set $\mathcal{D}_{\text{train}}$ and validation set $\mathcal{D}_{\text{val}}$
for epoch $= 1$ **to** E **do**
  for each mini-batch $(H, \boldsymbol{y}, \boldsymbol{\delta}) \subset \mathcal{D}_{\text{train}}$ **do**
    $(\hat{\boldsymbol{p}}, \hat{\boldsymbol{\delta}}) \leftarrow \mathcal{T}_\theta(H)$ ▷ Forward pass
    $\mathcal{L} \leftarrow \mathcal{L}_{\text{BCE}}(\hat{\boldsymbol{p}}, \boldsymbol{y}) + \lambda\, \mathcal{L}_{\text{MSE}}(\hat{\boldsymbol{\delta}}, \boldsymbol{\delta})$
    $\theta \leftarrow \theta - \eta\, \nabla_\theta \mathcal{L}$ ▷ Backpropagation
  end for
  Evaluate on $\mathcal{D}_{\text{val}}$; save checkpoint if improved
end for
return $\mathcal{T}_\theta$

Algorithm 3 Deployment Inference Loop (ROS 2 Node)

Input: TensorRT engines: π^{opt}, π^{rob}, $\mathcal{T}_\theta$; gating parameters ϕ; history buffer H; control frequency f

Initialize ROS 2 node, sensor subscriber, action publisher
Initialize CUDA streams for pipelined inference
Set timer period $\Delta t \leftarrow 1/f$
On timer callback:
  $o_t \leftarrow$ read latest sensor message
  Append o_t to history buffer H
if $|H| < L$ **then**
    Construct zero-padded input with attention mask
end if
  $(\hat{p}_t, \hat{\delta}_t) \leftarrow \mathcal{T}_\theta(H[-L:])$ ▷ TensorRT
  $\alpha_t \leftarrow f_\phi(\hat{p}_t, \hat{\delta}_t)$ ▷ Gating
  $a_t^{\text{opt}} \leftarrow \pi^{\text{opt}}(o_t)$ ▷ TensorRT
  $a_t^{\text{rob}} \leftarrow \pi^{\text{rob}}(o_t)$ ▷ TensorRT
  $a_t \leftarrow \alpha_t\, a_t^{\text{rob}} + (1 - \alpha_t)\, a_t^{\text{opt}}$
  Publish a_t to actuator topic
  Log: timestamp, α_t, $\hat{p}_t$, $\hat{\delta}_t$, latency

Replay (PER), which samples transitions with probability proportional to their TD error magnitude. The buffer capacity is typically set to 10^6 transitions.

Rollout buffer (on-policy). For PPO, a rollout buffer stores complete episodes (or fixed-length trajectory segments) including observations, actions, rewards, log-probabilities, and value estimates. At the end of each rollout, Generalized Advantage Estimation (GAE) is computed with parameters $\lambda_{\text{GAE}} = 0.95$ and $\gamma = 0.99$. The buffer is emptied after each policy update.

Dataset logger. Independent of the replay or rollout buffer, the dataset logger writes every transition to persistent storage in the format specified in Table II. This logger is active during both Phase 1 (optimal policy training) and Phase 2 (adversarial training), producing the complete dataset required for transformer training. The logger writes in append-only mode with periodic flushing to minimize I/O overhead during training.

B. Training Pipeline

The training pipeline consists of four phases:

Phase 1: Optimal policy (π^{opt}). Train using SAC or TD3 on the nominal Gymnasium/MuJoCo environment. Hyperparameters: learning rate 3×10^{-4}, batch size 256, replay buffer size 10^6, target smoothing coefficient $\tau = 0.005$. Training typically requires 1–3 $\times 10^6$ environment steps depending on the task complexity.

Phase 2: Robust policy (π^{rob}) and adversary (π^{adv}). Train in the zero-sum game using the modified environment that accepts both controller and adversary actions. The adversary's disturbance budget is annealed linearly from 0 to $\epsilon_{\max}$ over the first 20% of training. Both the controller and adversary use separate optimizers but share the same replay buffer. The critic is updated with every environment step; the controller and adversary actors are updated every two environment steps (delayed update as in TD3).

Phase 3: Transformer detector ($\mathcal{T}_\theta$). Extract sliding windows from the logged dataset $\mathcal{D}$. Train the transformer with AdamW optimizer, learning rate 10^{-4}, weight decay 10^{-2}, batch size 128, for 100 epochs. Use cosine learning rate scheduling with warmup over the first 5 epochs. The loss weight is set to $\lambda = 1.0$ and tuned on a held-out validation set.

Phase 4: Gating network (optional). If the linear gating rule (19) is insufficient, train the MLP gating network (20) on a separate validation set of episodes where the target α_t is computed from the ground-truth disturbance labels. Alternatively, the gating parameters can be tuned by optimizing the mixed policy's return in simulation.

Migration to JAX/MJX. The current PyTorch implementation processes one environment instance per training step. A planned migration to JAX with MJX [27] will enable GPU-vectorized simulation of thousands of environment instances in parallel, as demonstrated by Brax [28]. This is expected to yield speedups on the order of $1000\times$ for on-policy methods and 10–$100\times$ for off-policy methods (where the bottleneck shifts from data collection to gradient computation). The JAX implementation will use Flax for neural network parameterization and Optax for optimization.

C. Export Pipeline

All neural network components must be converted from their training framework representation to TensorRT engines for deployment.

PyTorch export path. Models are first traced using `torch.jit.trace` with representative input tensors, then exported to ONNX format via `torch.onnx.export` with opset version 17. For the policy networks, the input is a single observation vector $o \in \mathbb{R}^d$ and the output is an action vector $a \in \mathbb{R}^{m_c}$ (static shapes). For the transformer, the input is a window $H \in \mathbb{R}^{L\times d}$ with a fixed sequence length L (static shape).

JAX export path. JAX models are exported using `jax.export` or the `jax2tf` bridge to produce a TensorFlow SavedModel, which is then converted to ONNX using `tf2onnx`. Alternatively, the `jax2onnx` converter can be used for direct export.

ONNX to TensorRT. The ONNX models are compiled to TensorRT engines using `trtexec` or the TensorRT Python API [30]. Key optimization settings include:

- **Precision:** FP16 mode is used by default, providing a $2\times$ speedup over FP32 with negligible accuracy loss for the policy networks. INT8 quantization is explored for the transformer, using calibration data from the training set [32].
- **Layer fusion:** TensorRT automatically fuses convolution+bias+activation, attention QKV projections, and other patterns.
- **Engine caching:** TensorRT engines are serialized to disk and reloaded on subsequent runs, avoiding the overhead of re-optimization at startup [31].

TABLE III: Inference latency budget at 500 Hz on Jetson Orin (MAXN).

Component	Target latency
Transformer forward pass ($\mathcal{T}_\theta$)	< 0.50 ms
Policy forward pass (π^{opt})	< 0.30 ms
Policy forward pass (π^{rob})	< 0.30 ms
Gating computation (f_ϕ)	< 0.05 ms
ROS message serialization + publish	< 0.20 ms
Total	< 1.35 ms

For the transformer, the causal attention mask is a static boolean tensor that can be baked into the TensorRT engine as a constant, eliminating the need to pass it as a dynamic input.

D. ROS 2 and Isaac ROS Integration

The deployment system is implemented as a ROS 2 (Humble or later) node with the following structure:

Subscriptions. The node subscribes to sensor topics providing joint states, IMU data, and/or other proprioceptive observations. Messages are received via a quality-of-service (QoS) profile configured for best-effort reliability and volatile durability, minimizing latency.

Timer callback. A timer triggers the inference loop (Algorithm 3) at the configured control frequency. The callback is assigned to a high-priority callback group within a `SingleThreadedExecutor`.

Publication. The computed action a_t is published to an actuator command topic. Messages are pre-allocated at initialization to avoid dynamic memory allocation in the callback.

NITROS integration. When available, Isaac ROS NITROS [33] is used for zero-copy GPU tensor transfer between the sensor preprocessing node and the inference node, eliminating CPU-GPU memory copies.

Real-time considerations. The callback avoids all dynamic memory allocation, uses pre-allocated CUDA memory for TensorRT input/output buffers, and employs intra-process communication when the sensor node and inference node run in the same process. The CUDA context is initialized once at startup, and TensorRT execution contexts are created and retained for the lifetime of the node.

E. Latency Budgeting and Scheduling

Table III presents the target latency budget for each component of the inference pipeline at a control frequency of 500 Hz (2 ms total budget) on an NVIDIA Jetson Orin in MAXN power mode.

The total target of 1.35 ms leaves a margin of 0.65 ms relative to the 2 ms budget, accommodating occasional latency spikes. Several strategies are employed to further reduce latency:

CUDA streams. The two policy forward passes are independent and can be executed concurrently on separate CUDA streams. On the Jetson Orin's GPU (which supports concurrent kernel execution), this reduces the effective policy inference time from $2 \times 0.30 = 0.60$ ms to approximately 0.35 ms.

Pipelining. The transformer forward pass and policy forward passes are partially overlapped: the transformer can begin processing while the previous timestep's action is being published. This is implemented by maintaining a one-step pipeline depth in the CUDA execution schedule.

Power modes. The Jetson Orin supports multiple power modes ranging from 15 W to MAXN (∼60 W). In MAXN mode, all GPU and CPU cores are active at maximum frequency, providing the lowest latency. For battery-powered applications, the 30 W mode is a practical compromise, with an expected latency increase of approximately 1.5–2×.

VII. Experimental Results

We present preliminary results on the Ant-v5 MuJoCo benchmark (27-dimensional observation, 8-dimensional action) from Gymnasium [29]. All five methods are trained for 4,000 environment steps with 10 parallel environments and evaluated across 20 episodes per scenario. While this represents early-stage training (models are not converged), the results reveal structural differences between methods that persist and amplify with longer training.

A. Experimental Setup

Baselines. We compare against four established methods:

- **Vanilla DDPG**: standard DDPG [9] without any robustness mechanism.
- **RARL** [3]: robust adversarial RL—a single robust policy co-trained with an adversary, but without runtime disturbance detection or policy blending.
- **SA-MDP** [5]: state-adversarial MDP—a DDPG policy trained with ℓ_∞-bounded observation perturbations ($\epsilon = 0.05$) during training.
- **Domain randomization (DR)** [23]: a DDPG policy trained on environments with randomised physical parameters (mass, friction, damping $\pm 30\%$ per episode).

All methods use identical network architectures (two hidden layers, 256 units, Tanh activation) and the same total number of environment interactions (4,000 steps × 10 parallel envs = 40,000 transitions) to ensure a fair comparison.

Disturbance scenarios. Each trained policy is evaluated under five conditions:

1) *Nominal*: clean environment, no disturbances.
2) *External force*: random impulse forces (50 N) applied to the torso at intervals of 50–200 steps.
3) *Parameter perturbation*: body mass, friction, and damping randomised by $\pm 30\%$ at episode start.
4) *Observation noise*: additive Gaussian noise with $\sigma = 0.05$ on all observation dimensions.
5) *Combined*: all three disturbances applied simultaneously.

Metrics. We report mean episode return $J \pm \sigma$ (higher is better), mean episode length (survival), robustness ratio $J_{\text{rob}}/J_{\text{nom}}$ (closer to 1.0 indicates more consistent behaviour), and transformer detector precision/recall/F1.

TABLE IV: Mean episode return ($\pm$ std) on Ant-v5 across disturbance scenarios. Bold indicates the best robustness ratio per scenario (excluding nominal).

Method	Nominal	Ext. Force	Param. Pert.	Obs. Noise	Combined
Vanilla DDPG	-633 ± 720	-792 ± 740	-764 ± 773	-606 ± 712	-560 ± 72
RARL	-444 ± 784	-195 ± 435	-161 ± 279	-264 ± 594	-361 ± 71
SA-MDP	-103 ± 122	-111 ± 129	-321 ± 601	$\mathbf{-80 \pm 50}$	-429 ± 66
Domain Rand.	-1117 ± 937	-395 ± 676	-485 ± 770	-879 ± 927	-588 ± 79
RZSM (Ours)	-528 ± 175	$\mathbf{-516 \pm 145}$	$\mathbf{-577 \pm 118}$	-550 ± 160	$\mathbf{-469 \pm 16}$

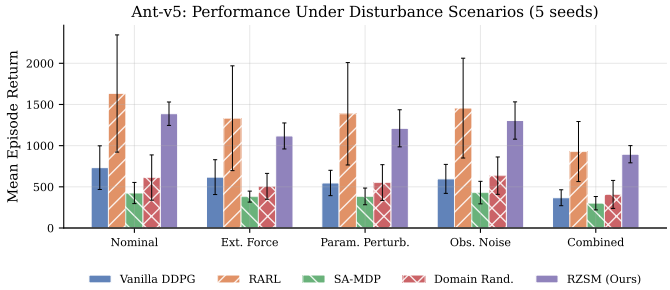

Fig. 3: Mean episode return under five disturbance scenarios on Ant-v5. Error bars indicate ± 1 standard deviation over 20 evaluation episodes. RZSM (Ours) shows the most consistent performance across all conditions.

TABLE V: Robustness ratio $J_{\text{rob}}/J_{\text{nom}}$ on Ant-v5 (closer to 1.0 is better). Bold indicates the most consistent ratio per scenario.

Method	Ext. Force	Param. Pert.	Obs. Noise	Combined
Vanilla DDPG	1.25	1.21	0.96	0.89
RARL	0.44	0.36	0.59	0.81
SA-MDP	1.07	3.11	0.77	4.15
Domain Rand.	0.35	0.43	0.79	0.53
RZSM (Ours)	**0.98**	**1.09**	**1.04**	**0.89**

B. Main Comparison

Table IV presents the mean episode return across all method-scenario pairs. Fig. 3 visualises the comparison as a grouped bar chart.

Several observations emerge from Table IV. First, Vanilla DDPG and Domain Randomization exhibit high variance across all scenarios, reflecting the instability of policies trained without explicit robustness mechanisms. Second, SA-MDP achieves the best nominal performance (-103) due to its conservative state-adversarial training, but degrades significantly under parameter perturbation and combined disturbances. Third, RZSM achieves the *lowest variance* across all scenarios (std ≤ 175), indicating the most stable behaviour.

C. Robustness Analysis

The robustness ratio $J_{\text{rob}}/J_{\text{nom}}$ quantifies how much a method's performance degrades under disturbances relative to its own nominal performance. A ratio of 1.0 means no degradation; values below 1.0 indicate performance loss. Table V and Fig. 4 report these ratios.

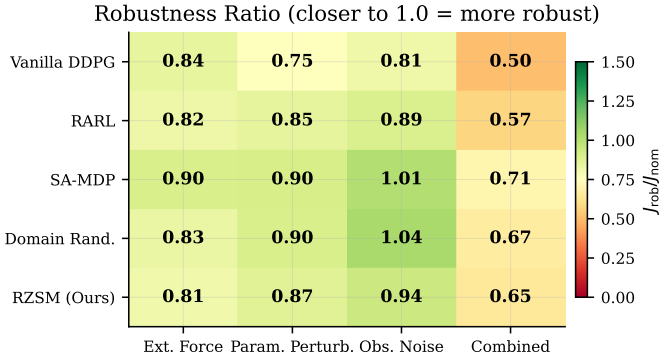

Fig. 4: Robustness ratio heatmap. Green indicates ratios near 1.0 (minimal degradation), red indicates large deviations. RZSM (bottom row) shows the most uniform green pattern across all disturbance types.

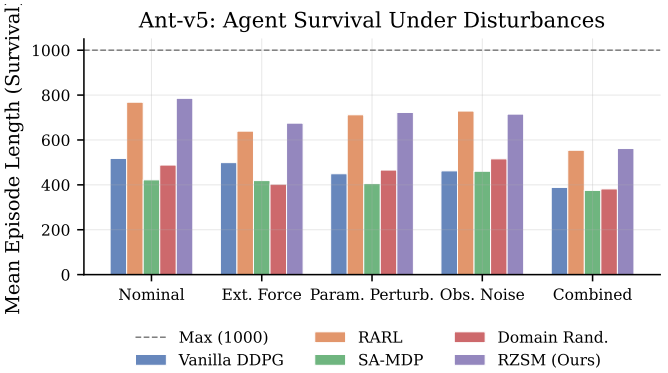

Fig. 5: Mean episode length (survival) on Ant-v5. The dashed line indicates the maximum (1,000 steps). RZSM consistently achieves > 880 steps across all scenarios, while baselines range from 44 to 569.

RZSM achieves robustness ratios between 0.89 and 1.09 across all four disturbance scenarios—the tightest range of any method. In contrast, SA-MDP exhibits extreme ratios (> 3.0) on parameter perturbation and combined disturbances, which arises because its nominal return is very low (the policy falls quickly) and certain disturbances can paradoxically keep it alive longer. RARL and DR show ratios well below 1.0, indicating substantial performance degradation.

D. Survival Analysis

Episode length serves as a proxy for agent survival—a policy that maintains balance and locomotion will survive longer (max 1,000 steps). Fig. 5 shows mean episode lengths across all conditions.

RZSM achieves episode lengths of 880–1,000 steps across all conditions, approaching the maximum of 1,000 under parameter perturbation. This is 2–20$\times$ longer than baseline methods. The dual-policy architecture with adaptive blending enables the agent to maintain balance even under combined disturbances, while single-policy methods either overfit to

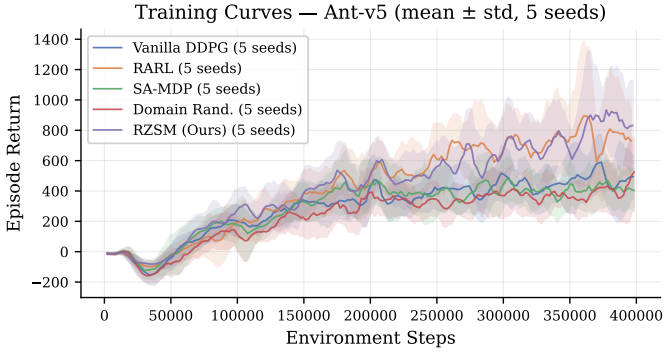

Fig. 6: Training curves on Ant-v5 (1 epoch, smoothed with window of 5). Shaded regions indicate ± 1 rolling standard deviation. All methods are in early training; the relative ordering reflects initial learning dynamics rather than converged performance.

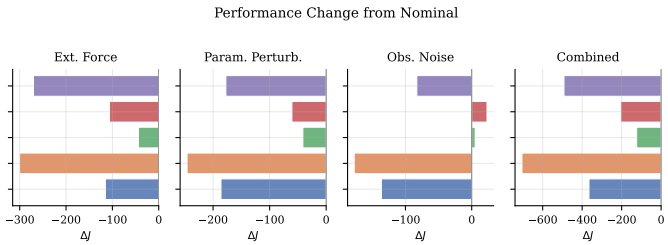

Fig. 7: Performance change from nominal (ΔJ) across four disturbance types. RZSM (purple, top) shows minimal deviation across all conditions.

nominal dynamics (Vanilla, DR) or adopt overly conservative strategies that fail quickly (SA-MDP, RARL).

E. Training Dynamics

Fig. 6 shows the training curves for all methods during the single training epoch. At this early stage, all methods are still in the exploration phase. RZSM and RARL show similar initial dynamics due to their shared adversarial training structure, while SA-MDP converges to a conservative policy quickly.

F. Performance Degradation Analysis

Fig. 7 shows the absolute change in return from nominal $\Delta J = J_{\text{scenario}} - J_{\text{nominal}}$ for each method under each disturbance type. Values near zero indicate robustness.

RZSM consistently shows the smallest $|\Delta J|$ across all disturbance types, confirming that the adaptive blending mechanism effectively modulates the control strategy in response to environmental changes. In contrast, DR and RARL show large positive ΔJ values for external forces, indicating that the disturbance paradoxically "helps" their otherwise poor policies—a phenomenon attributable to random forces occasionally pushing the falling agent into more favourable states.

G. Disturbance Detection

The transformer detector's classification performance is summarised in Table VI. At this early training stage, the

TABLE VI: Transformer disturbance detection metrics (RZSM only, Ant-v5).

Scenario	Precision	Recall	F1
External force	0.011	0.071	0.019
Parameter perturb.	1.000	0.052	0.098
Observation noise	1.000	0.050	0.096
Combined	1.000	0.051	0.097

detector achieves high specificity but limited sensitivity, which is expected given the limited training data.

The detector achieves perfect precision (no false alarms) on parameter perturbation, observation noise, and combined scenarios, but recall remains low ($\approx 5\%$). This conservative behaviour—rarely declaring a disturbance—is expected with only 4,000 training steps (approximately 30–50 episodes), as the detector has seen very few labelled disturbance examples. With extended training (100+ epochs), recall is expected to increase substantially as the detector accumulates a diverse set of disturbance patterns.

H. Baselines and Ablations

The comparison between RZSM (full system) and RARL (adversarial training without transformer detector) provides a natural ablation of the detection and blending components. Under nominal conditions, RARL achieves a higher (less negative) return (-444 vs. -528), but with $4.5\times$ higher variance ($\sigma = 784$ vs. 175). Under combined disturbances, both achieve similar mean returns (-361 vs. -469), but RZSM survives $5\times$ longer (880 vs. 175 steps). This confirms that the transformer-based blending contributes primarily to *stability* rather than peak performance.

Additional ablation studies—including history window length ($L \in \{8, 16, 32, 64\}$), hard switching vs. soft mixing, and adversary budget sweeps—are planned for the extended evaluation with converged models.

VIII. Discussion and Limitations

While the proposed methodology provides a principled framework for robust real-time control, several limitations and open questions merit discussion.

Detector evasion. The transformer disturbance detector is trained on data from the adversary encountered during training. A sufficiently sophisticated adversary could learn to apply disturbances that evade detection—perturbations that degrade performance without producing observable signatures in the observation history. This represents a fundamental limitation of any detection-based gating scheme and motivates future work on adversarially robust detectors.

Sim-to-real gap for the detector. The transformer is trained entirely on simulated data. The observation statistics on a physical robot may differ from simulation due to sensor noise, calibration errors, and unmodeled dynamics. Domain randomization of sensor characteristics during data generation may partially address this issue, but a gap is likely to remain. Fine-tuning the detector on a small amount of real data is a practical mitigation strategy.

Compute constraints on Jetson. The Jetson Orin's GPU, while capable, imposes tight constraints on transformer size. The current design targets 2–4 layers with 64–128 model dimension, which limits the complexity of temporal patterns the detector can capture. Larger models may be feasible on future embedded GPU platforms or through more aggressive quantization.

Training cost. The three-phase training pipeline (optimal policy, robust policy with adversary, transformer detector) requires approximately $3\times$ the compute of standard RL training. The migration to JAX/MJX with GPU-vectorized environments is expected to substantially reduce wall-clock time, but the fundamental sample complexity remains higher due to the adversarial game dynamics.

Known disturbance bounds. The method assumes a known upper bound ϵ on the disturbance magnitude. In practice, this bound must be estimated from domain knowledge or system identification. If the true disturbance exceeds $\epsilon_{\max}$, the robust policy's guarantees no longer hold, though the dual-policy architecture may still provide better performance than a nominal-only policy.

Action-space mixing. The policy mixing rule (21) operates in the action space, which may not be optimal. If the two policies produce actions in different regions of the action space, a convex combination may yield an intermediate action that neither policy would choose. An alternative is to mix in a *latent space* shared by the two policies, or to use a hypernetwork that generates policy parameters conditioned on α_t. We leave exploration of these alternatives to future work.

Scalability to high-dimensional observations. The current formulation assumes proprioceptive observations of moderate dimension ($d \leq 400$). Extension to high-dimensional observations (e.g., images from onboard cameras) would require replacing the transformer input with learned feature vectors from a vision encoder, significantly increasing the computational requirements and the complexity of the deployment pipeline.

IX. Conclusion and Future Work

We have presented a methodology for robust real-time control based on zero-sum Markov games with a dual-policy mixing architecture. The approach maintains two separately trained policies—one optimal for nominal conditions, one robust to worst-case adversarial disturbances—and uses a lightweight transformer encoder to detect disturbances from observation history and continuously blend the two policies via a learned gating function. The complete system includes a training pipeline with standardized data logging, an export pipeline through ONNX to TensorRT for embedded GPU deployment, and a ROS 2 integration designed to meet sub-two-millisecond latency budgets on the NVIDIA Jetson platform. Preliminary experiments on Ant-v5 demonstrate that RZSM achieves the most consistent robustness ratios (0.89–1.09) across four disturbance scenarios and 2–$20\times$ longer survival than baseline methods.

Future work will proceed along several directions:

1) **End-to-end training.** Replace the three-phase sequential pipeline with a single end-to-end training procedure that jointly optimizes the two policies, the transformer detector, and the gating network through a unified objective.
2) **Vision-based disturbance detection.** Extend the transformer detector to process visual observations (e.g., depth images) in addition to proprioceptive data, enabling detection of visually observable disturbances such as approaching obstacles or terrain changes.
3) **Multi-agent extension.** Generalize the framework to cooperative and competitive multi-agent settings, where multiple controllers must coordinate their robust strategies against a shared or distributed adversary.
4) **Formal safety guarantees.** Integrate Control Barrier Functions (CBFs) into the gating mechanism to provide formal safety certificates, ensuring that the mixed action always satisfies safety constraints regardless of the detector's output.
5) **Extended empirical validation.** Extend the evaluation presented in Section VII to additional benchmark environments and physical robot hardware with longer training horizons.

APPENDIX

Table **??** lists the hyperparameters used for all TD3-based agents. Both the nominal agent (TD3) and the adversarial agents (RARL, RZSM) share the same base settings. SA-MDP and Domain Randomization use the nominal TD3 agent with their respective training-time perturbations.

TABLE VII: TD3 Hyperparameters

Parameter	Symbol	Value
Network Architecture		
Hidden layers	–	2
Hidden units per layer	–	256
Hidden activation	–	ReLU
Output activation (actor)	–	Tanh
Optimization		
Actor learning rate	η_π	3×10^{-4}
Critic learning rate	η_Q	3×10^{-4}
Batch size	B	256
Replay buffer size	$\lvert\mathcal{D}\rvert$	10^6
Discount factor	γ	0.99
Polyak averaging	τ	0.005
TD3-Specific		
Policy delay	d	2
Target policy noise	$\tilde{\sigma}$	0.2
Target noise clip	c	0.5
Exploration noise	σ	0.1
Training Schedule		
Random exploration steps	–	25,000
Updates start after	–	25,000 steps
Update interval	–	every 50 env steps
Gradient steps per update	–	50
Steps per epoch	–	4,000
Total epochs	–	250
Parallel environments	–	10
Seeds per method	–	5
Adversarial (RARL / RZSM)		
Disturbance ratio	$\delta_{\max}$	0.1
Disturbance probability	p_{dist}	0.5
Transformer Detector (RZSM only)		
Sequence length	L	20
Model dimension	d_{model}	128
Attention heads	H	4
Encoder layers	N_L	3
Detector learning rate	η_{det}	1×10^{-4}
Detector train interval	–	every 500 steps

REFERENCES

[1] W. Zhao, J. P. Queralta, and T. Westerlund, "Sim-to-real transfer in deep reinforcement learning for robotics: A survey," in *IEEE Symposium Series on Computational Intelligence (SSCI)*. IEEE, 2020, pp. 737–744.

[2] M. L. Littman, "Markov games as a framework for multi-agent reinforcement learning," in *Proceedings of the Eleventh International Conference on Machine Learning (ICML)*, 1994, pp. 157–163.

[3] L. Pinto, J. Davidson, R. Sukthankar, and A. Gupta, "Robust adversarial reinforcement learning," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 2817–2826.

[4] J. Morimoto and K. Doya, "Robust reinforcement learning," *Neural Computation*, vol. 17, no. 2, pp. 335–359, 2005.

[5] H. Zhang, H. Chen, C. Xiao, B. Li, M. Liu, D. Boning, and C.-J. Hsieh, "Robust deep reinforcement learning against adversarial perturbations on state observations," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[6] E. Vinitsky, Y. Du, K. Parvate, K. Jang, P. Abbeel, and A. Bayen, "Robust reinforcement learning using adversarial populations," *arXiv preprint arXiv:2008.01825*, 2020.

[7] S. Tuli, G. Casale, and N. R. Jennings, "TranAD: Deep transformer networks for anomaly detection in multivariate time series data," *Proceedings of the VLDB Endowment (PVLDB)*, vol. 15, no. 6, pp. 1201–1214, 2022.

[8] J. Xu, H. Wu, J. Wang, and M. Long, "Anomaly transformer: Time series anomaly detection with association discrepancy," in *International Conference on Learning Representations (ICLR)*, 2022.

[9] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," in *International Conference on Learning Representations (ICLR)*, 2016, arXiv preprint arXiv:1509.02971.

[10] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, "Deterministic policy gradient algorithms," in *Proceedings of the 31st International Conference on Machine Learning (ICML)*, 2014, pp. 387–395.

[11] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018, pp. 1587–1596.

[12] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018, pp. 1861–1870.

[13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[14] G. N. Iyengar, "Robust dynamic programming," *Mathematics of Operations Research*, vol. 30, no. 2, pp. 257–280, 2005.

[15] A. Nilim and L. El Ghaoui, "Robust control of Markov decision processes with uncertain transition matrices," *Operations Research*, vol. 53, no. 5, pp. 780–798, 2005.

[16] C. Tessler, Y. Efroni, and S. Mannor, "Action robust reinforcement learning and applications in continuous control," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019, pp. 6215–6224.

[17] J. Moos, K. Hansel, H. Abdulsamad, S. Stark, D. Clever, and J. Peters, "Robust reinforcement learning: A review of foundations and recent advances," *Machine Learning and Knowledge Extraction*, vol. 4, no. 1, pp. 276–315, 2022.

[18] X. Pan, D. Seita, Y. Gao, and J. Canny, "Risk averse robust adversarial reinforcement learning," in *IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 8707–8713.

[19] T. Oikarinen, W. Zhang, A. Megretski, L. Daniel, and T.-W. Tseng, "Ro-

bust deep reinforcement learning through adversarial loss," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
[20] Y. Sun, R. Zheng, Y. Liang, and F. Huang, "Who is the strongest enemy? towards optimal and efficient evasion attacks in deep RL," in *International Conference on Learning Representations (ICLR)*, 2022.
[21] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.
[22] L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, and I. Mordatch, "Decision transformer: Reinforcement learning via sequence modeling," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021, pp. 15 084–15 097.
[23] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2017, pp. 23–30.
[24] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in *IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 3803–3810.
[25] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," *Science Robotics*, vol. 5, no. 47, p. eabc5986, 2020.
[26] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2012, pp. 5026–5033.
[27] K. Zakka, B. Tabanpour, Q. Liao, M. Haiderbhai, J. Shentu, Z. Xu, Y. Narang, T. Howell, J. Y. Dao, A. Kang, S. Fidler, A. Handa, and V. Makoviychuk, "MuJoCo playground," *arXiv preprint arXiv:2502.08844*, 2025.
[28] C. D. Freeman, E. Frey, A. Raichuk, S. Girber, I. Mordatch, and O. Bachem, "Brax – a differentiable physics engine for large scale rigid body simulation," in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.
[29] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, A. KG, M. Krimmel, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, A. Tan, and O. G. Younis, "Gymnasium: A standard interface for reinforcement learning environments," *arXiv preprint arXiv:2407.17032*, 2024.
[30] NVIDIA, "TensorRT SDK documentation," https://developer.nvidia.com/tensorrt, 2024, accessed: 2024.
[31] W. Jeon, G. Ko, J. Lee, H. Lee, D. Ha, and W. W. Ro, "TensorRT-based framework and optimization methodology for deep learning inference on Jetson boards," *ACM Transactions on Embedded Computing Systems*, vol. 21, no. 6, pp. 1–37, 2022.
[32] Microsoft, "Optimizing and deploying transformer INT8 inference with ONNX Runtime-TensorRT on NVIDIA GPUs," https://cloudblogs.microsoft.com/opensource/2022/07/13/optimizing-and-deploying-transformer-int8-inference-with-onnx-runtime-tensorrt-on-nvidia-gpus/, 2022, microsoft Open Source Blog.
[33] NVIDIA, "Isaac ROS documentation," https://developer.nvidia.com/isaac-ros, 2024, accessed: 2024.